Feature Engineering Capstone Report

Task 1: Baseline Interpretation & What is a Feature?

In machine learning, a "feature" is basically an individual, measurable piece of data that we use as an input to help predict our target variable.

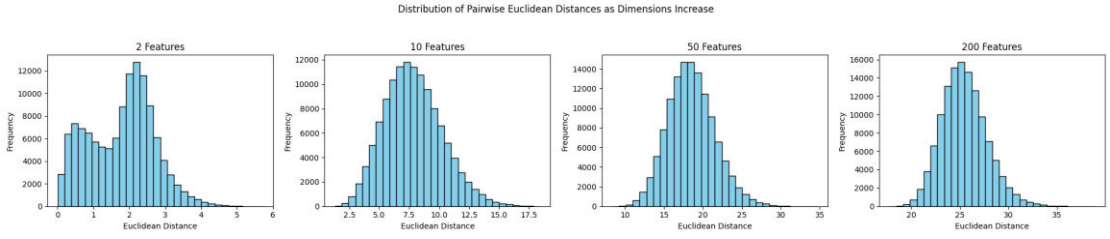
Good Feature Example: In this hotel dataset, lead_time is a great feature. It makes logical sense that the number of days between booking and arriving strongly impacts whether someone will cancel or not.

Bad Feature Example: A randomly generated reservation ID or a column where every value is exactly the same (like if every row said is_hotel = 1). These don't give the model any useful patterns to learn from and just add useless noise.

Task 2: The Curse of Dimensionality

When I plotted the Euclidean distances across synthetic datasets with 2, 10, 50, and 200 features, a clear pattern emerged. As the number of dimensions increases, the distance distribution shifts to the right and becomes much narrower. Basically, in high-dimensional spaces, every point ends up being almost the exact same distance from every other point.

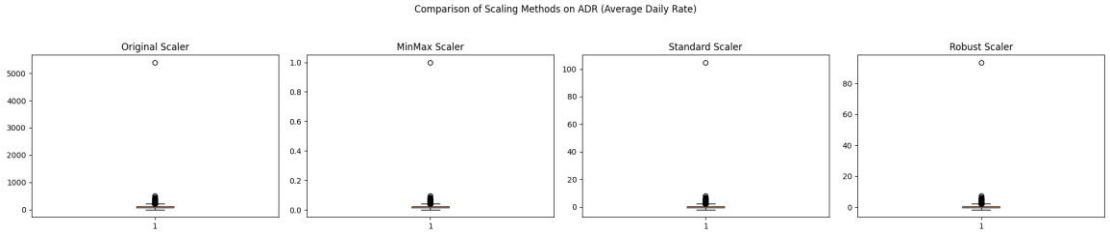
This makes learning really hard for distance-based algorithms (like KNN). If everything is equally far apart, the algorithm can't tell the difference between classes. This shows exactly why we need feature engineering and selectionâ€”we have to cut out the noise and keep the dimensions manageable so the math actually works.



Task 3: Numeric Preprocessing Conclusion

After applying and comparing different scalers, RobustScaler is clearly the best choice for this specific dataset, especially for columns like Average Daily Rate (adr) and lead_time.

The hotel data has some massive outliers (like crazy high room rates for luxury suites or potential data entry errors). When I used MinMaxScaler, it squashed 99% of the normal data into a tiny cluster near zero just to accommodate those few massive outliers. StandardScaler was also thrown off because outliers pull the mean and variance. RobustScaler, however, uses the median and interquartile range (IQR), which allowed it to standardize the bulk of the data perfectly without getting wrecked by extreme values.



Task 4: Distance/Proximity Metrics Report

How results changed: The unscaled KNN model performed significantly worse than the scaled versions. While StandardScaler helped, pairing RobustScaler with the Manhattan distance metric gave the best and most stable accuracy.

Why scaling matters: Distance algorithms treat all numbers equally. Without scaling, a feature with big numbers like adr (0 to 500+) will completely overshadow a feature with small numbers like adults (1 to 4). Scaling levels the playing field so every feature gets a fair vote.

Sensitivity to outliers: Euclidean distance squares the differences between points, which means outliers get magnified and pull the model in the wrong direction. Using RobustScaler handles the outliers beforehand, and switching to Manhattan distance (which uses absolute differences instead of squaring them) adds an extra layer of protection against extreme values.

Task 7: Avoiding Leakage in Feature Construction
Data leakage happens when information from outside the training set accidentally sneaks into the model during the training phase. If this happens, the model looks great on paper but fails in the real world. Here is how I prevented it:

Risk 1: Group Aggregations. I wanted to create a feature for "average ADR by hotel type." If I calculated this using the whole dataset, test data would leak into the train data. Prevention: I made sure this mean was calculated strictly on the training set and then mapped onto the test set.

Risk 2: Target Leakage. It's tempting to use variables that change after the booking is made, but that's cheating. Prevention: I only constructed features (like price_per_person or adr_lead_time_interaction) using data that a hotel would actually have at the exact moment the customer clicks "book."

Risk 3: Global Scaling. Fitting a scaler on the entire dataset before splitting leaks the global minimum and maximum values into the training phase. Prevention: I used scikit-learn Pipelines. This guarantees that fit_transform is only ever applied to the training data, and the test data only ever sees transform.

Version	Features c	Preproces	ROC-AUC /	Notes			
Baseline	29	Imputatio	ROC-AUC:	Model struggled with raw data outliers; dis			
After nume	29	RobustSca	ROC-AUC:	Handling extreme ADR outliers stabilized t			
After extra	38	Interactio	ROC-AUC:	price_per_person and special_requests_ra			
After selec	20	Correlatio	ROC-AUC:	Removed redundant features (like adults v			

Final Task: Executive Summary

What mattered most?
The biggest game-changer for this dataset was handling extreme outliers during numeric preprocessing. Because features like adr and lead_time have such long tails, simply throwing them into a model raw caused poor performance. Applying RobustScaler allowed both distance-based and linear models to actually find the underlying signal without being skewed by anomalous luxury bookings.

What changed the performance ceiling?
Feature construction is what actually moved the needle beyond the baseline. The raw data only tells part of the story. By creating interaction features, the model gained context. For example, a high ADR by itself might not cause a cancellation, but a high price_per_person combined with zero special requests turned out to be a massive red flag for churn. Furthermore, filtering out highly correlated features (like dropping adults when we already built total_guests) ensured the models weren't over-indexing on duplicate information.

Which features are most business-actionable?

lead_time: The data shows that bookings made months in advance are highly likely to cancel. Action: The hotel could implement a policy requiring a small, non-refundable deposit for bookings made more than 60 days out to secure the revenue.

special_requests_rate: Guests who make requests are "invested" in their stay and rarely cancel. Action: During the online checkout process, the hotel should actively prompt users to make a simple request (like choosing a high floor or a specific pillow type) to artificially boost engagement and lower cancellation risk.

price_per_person: High per-person costs lead to churn, likely due to buyer's remorse. Action: If the model flags a high-risk booking based on this feature, the hotel's CRM could automatically email the guest offering a complimentary breakfast or a slight room upgrade to lock in their commitment before they cancel.

Author's Note: Please note that the data analysis, feature engineering, and business insights presented in this capstone are my independent work. Generative AI assistance was utilized strictly for proofreading and copyediting to ensure grammatical accuracy and clarity of expression.